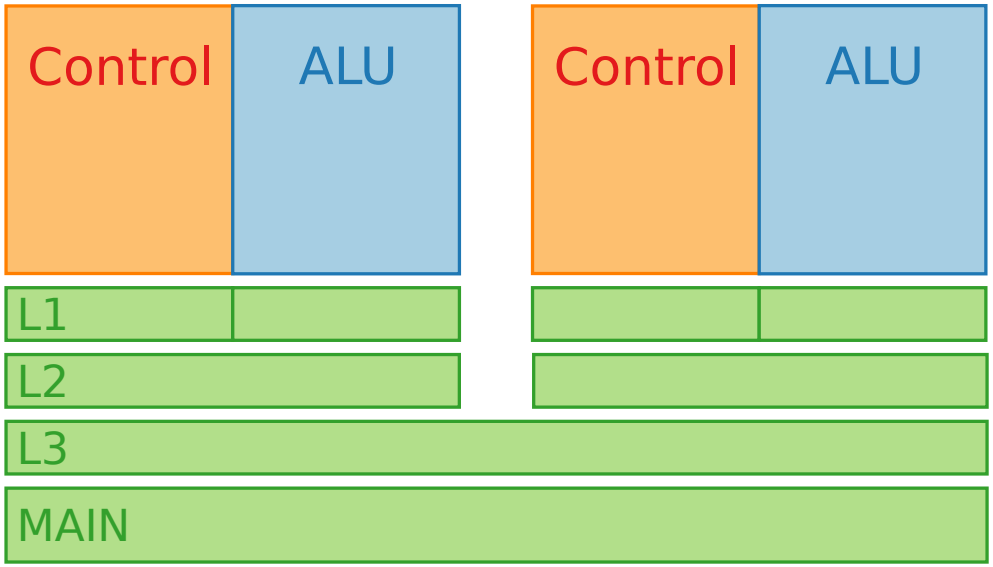
Thread Level Parallelism

IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

Why do we need Thread Level Parallelism ?

- Limits of ILP
- From 1 inefficient processors to multiple efficient processors
- Growing interest in high-end servers
- More data-intensive applications
- Improved understanding on how to leverage multiprocessors
  - Embarassingly parallel problems in Scientific / Engineering codes
  - Request-level parallelism
- Leveraging design investment by replication

Multicore



- Add CPU cores on a *single* chip
- Last Level Cache (LLC) is shared between cores
- Potential linear increasing of computing capacity
- Multiple threads : multiple instance of a program running simultaneously

Process

A process is the code, data and status of a program that is running on a computer

- Process State (ready, running, waiting,...)
- Program Counter
- CPU registers
- Scheduling information
- Memory management (Address space)

Process

A process is the code, data and status of a program that is running on a computer

- Process State (ready, running, waiting,...)
- Program Counter
- CPU registers
- Scheduling information
- Memory management (Address space)

Process

A process is the code, data and status of a program that is running on a computer

- Process State (ready, running, waiting,...)
- Program Counter
- CPU registers
- Scheduling information
- Memory management (Address space)

Process

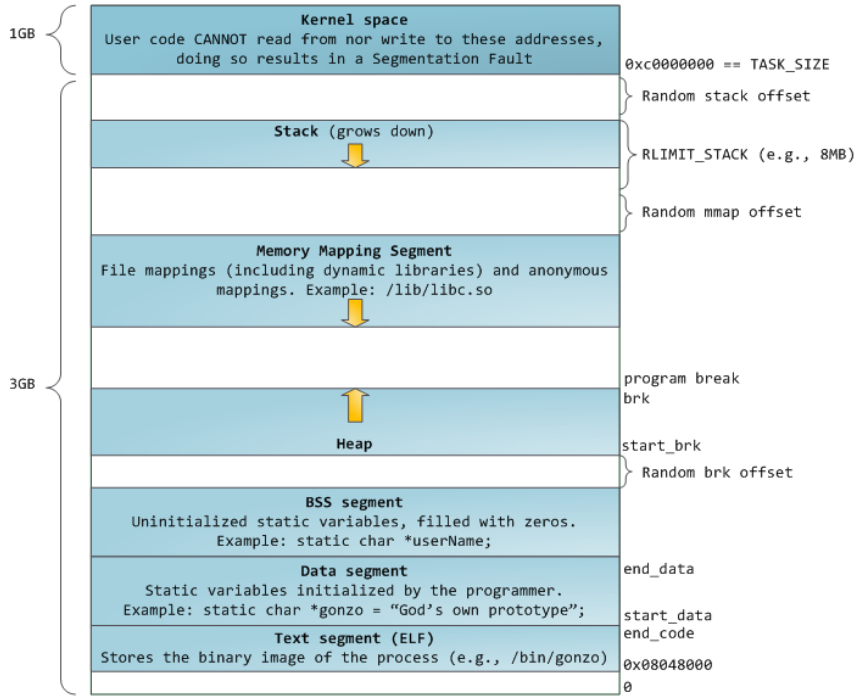
A process is the code, data and status of a program that is running on a computer

- Process State (ready, running, waiting,...)
- Program Counter
- CPU registers
- Scheduling information
- Memory management (Address space)

Process

A process is the code, data and status of a program that is running on a computer

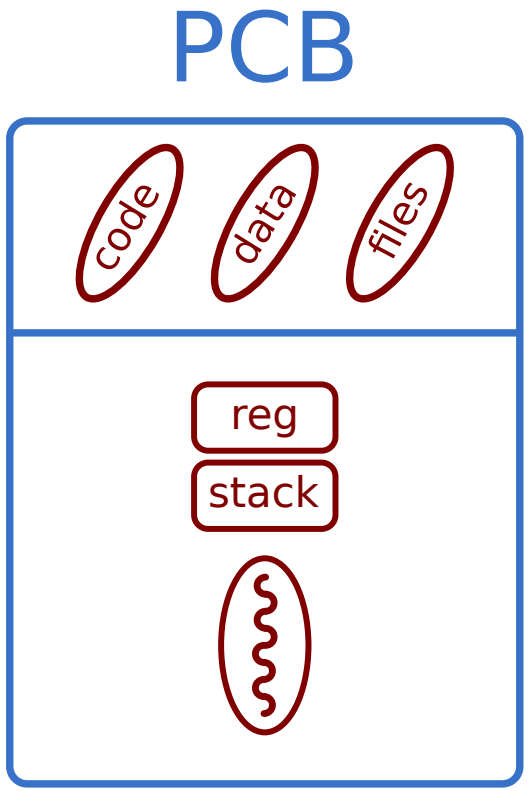
- Process State (ready, running, waiting,...)
- Program Counter
- CPU registers
- Scheduling information
- Memory management (Address space)



Process

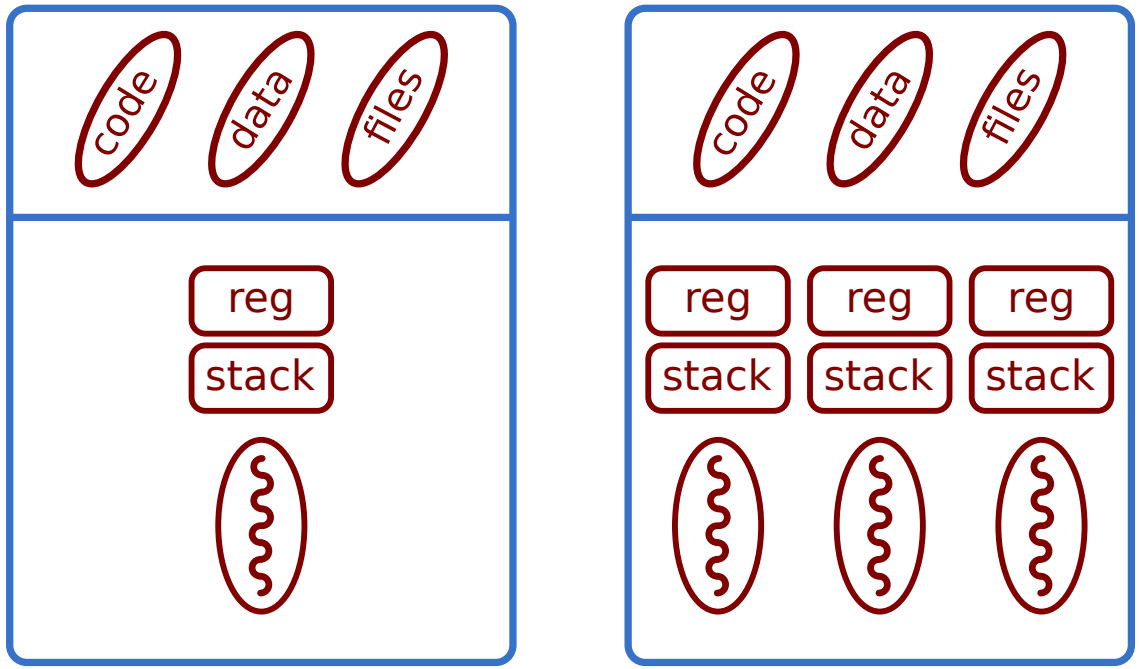
A process is the code, data and status of a program that is running on a computer

- Process State (ready, running, waiting,...)
- Program Counter
- CPU registers
- Scheduling information
- Memory management (Address space)



Process vs Thread

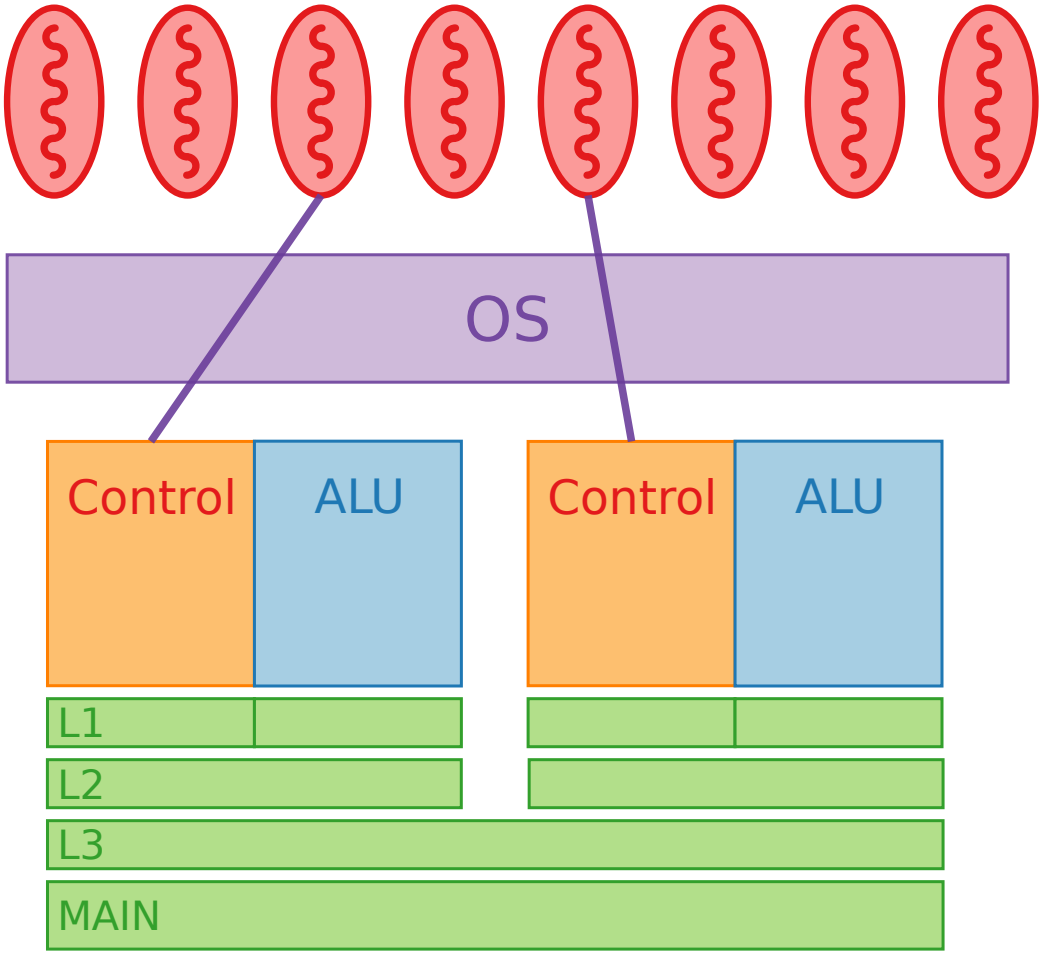
A process can consist of multiple threads.



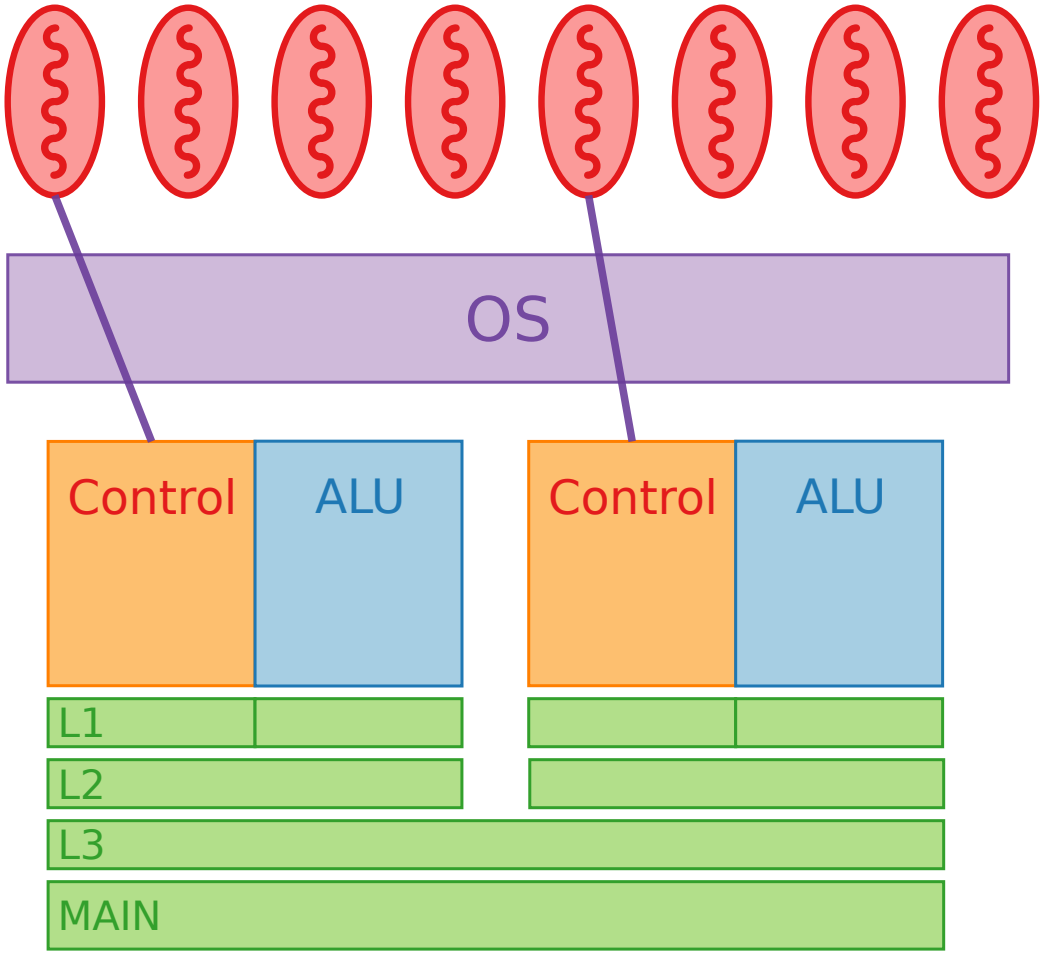
Disambiguation: Thread

- Software POV:
  - On a standard OS, an instance of a program is a process
  - Each process has one ore more threads
  - The OS is in charge of the scheduling
- Hardware POV:
  - On a single core CPU w/o SMT, only 1 active thread
  - On multicore and SMT, a finite number of parallel active threads

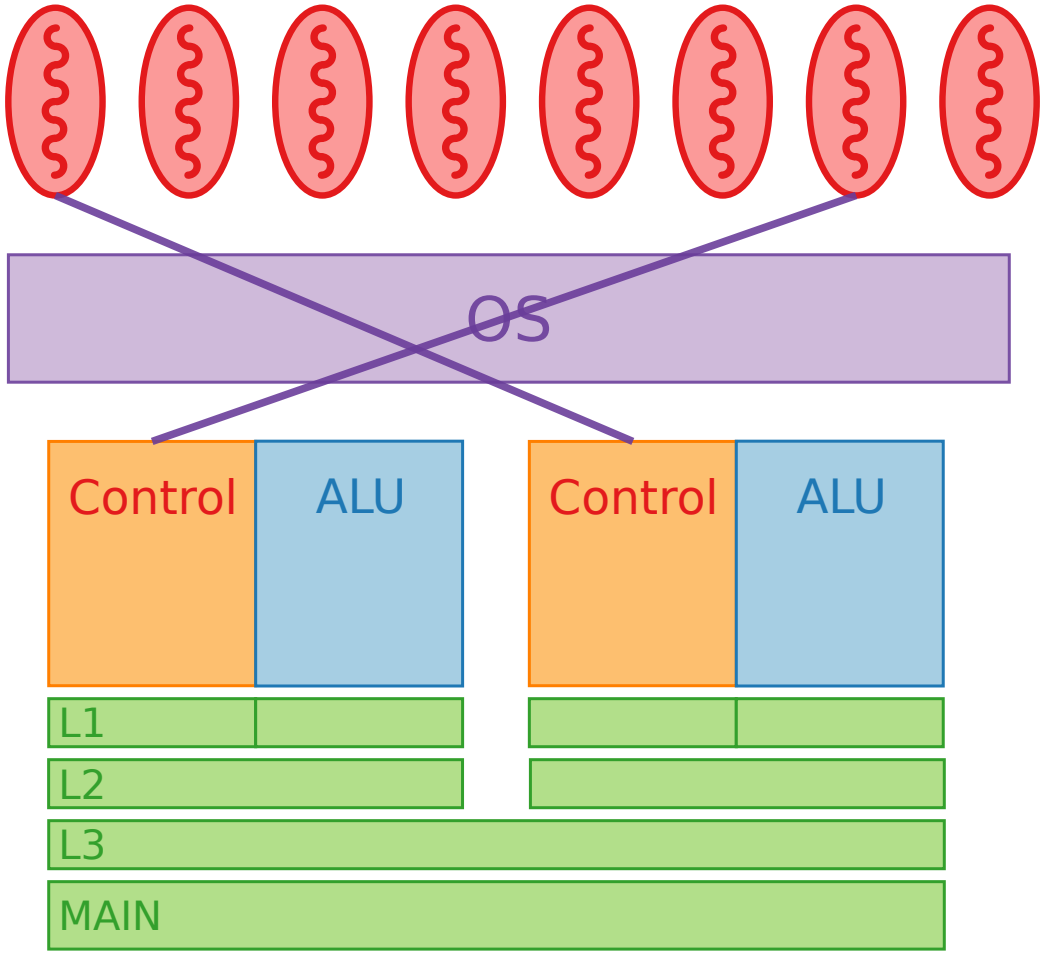
Scheduling



Scheduling



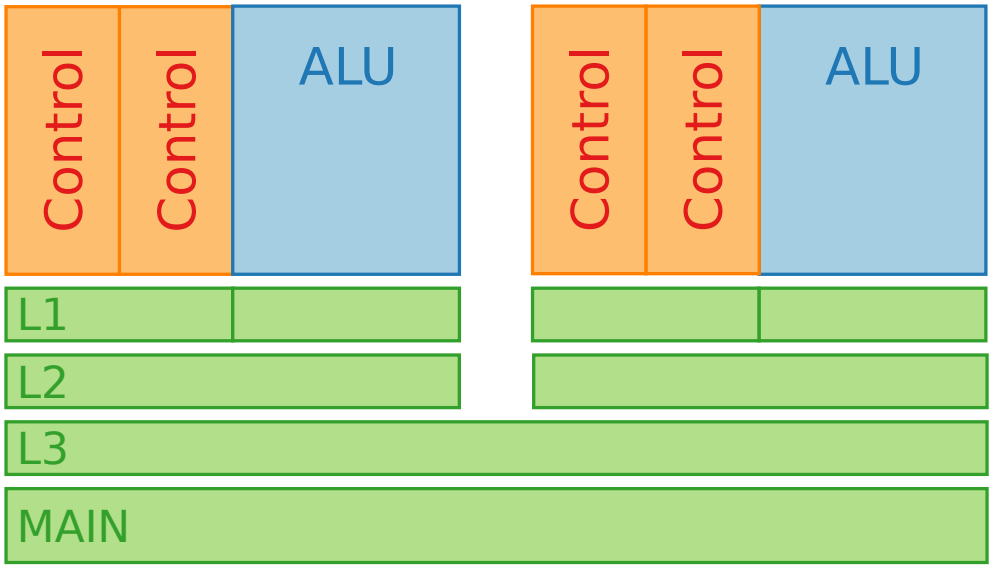
Scheduling



Scheduling

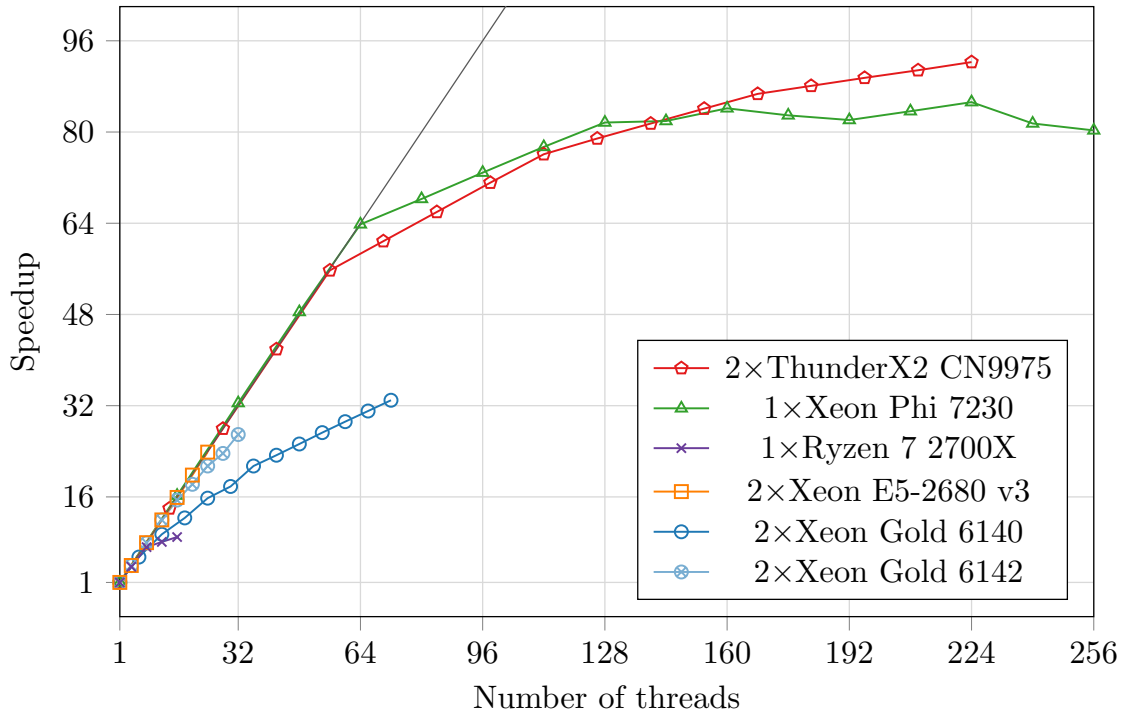
- CPU scheduling: managing processes and threads.
- A CPU core can only execute one thread at a time.
- Threads must be run one after the other.
- Run-to-Completion Scheduling
- Preemptive Scheduling (SJF (shortest job first) + priority)

Simultaneous Multi Threading (SMT)



- Known as "Hyperthreading" which is Intel's own SMT implementation
- Multiple instruction threads (here 2) are processed on each core
- Sublinear increasing of computing capacity, resources are shared

Multicore vs SMT: sub-linearity

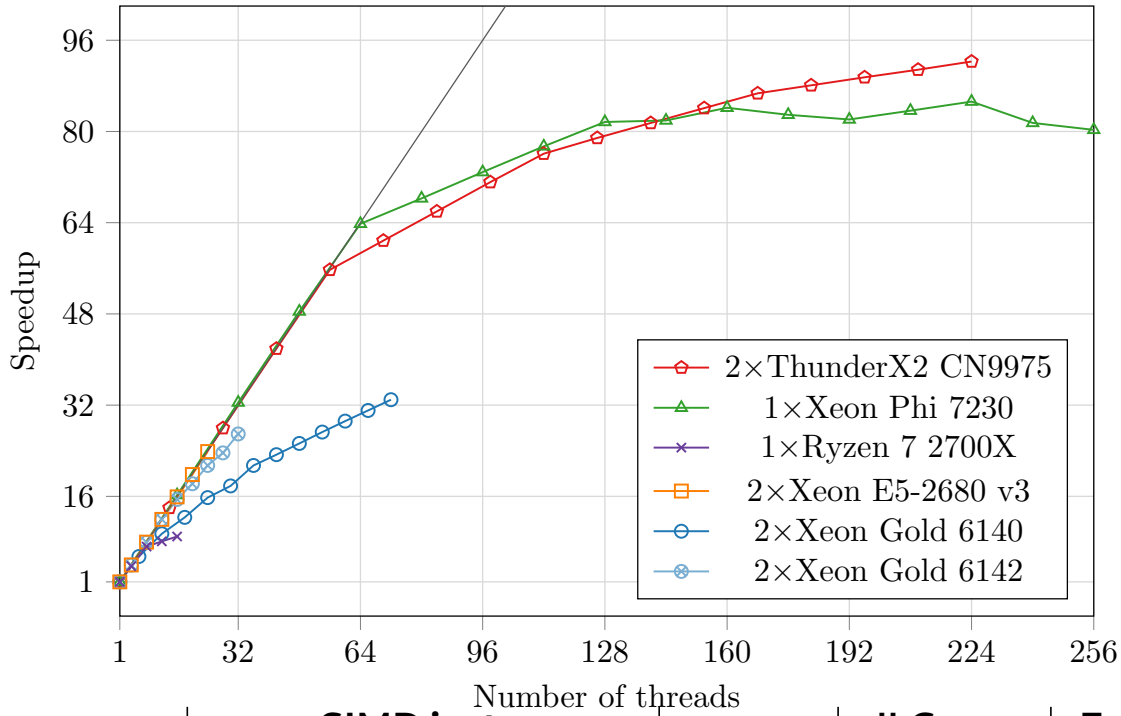


Question

- What is the number of physical cores ?
- What is the number of logical cores ?

`https://doi.org/10.1016/j.softx.2019.100345`

Multicore vs SMT: sub-linearity



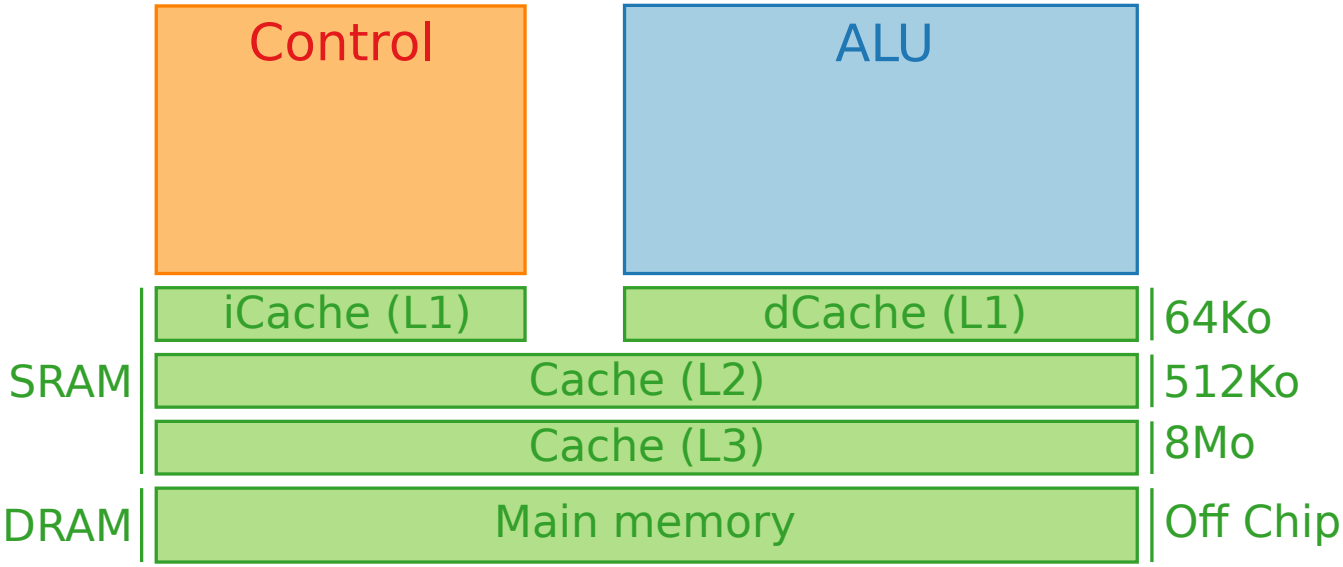
CPU		SIMD instr.		# Proc.	# Cores per Proc.	Freq. (GHz)	SMT	Turbo Boost
		Name	Size					
ARM	ThunderX2 CN9975	NEON	128-bit	2	28	2.00	4	✗
Intel	Xeon Phi 7230	AVX-512F	512-bit	1	64	1.30	4	✓
AMD	Ryzen 7 2700X	AVX2	256-bit	1	8	3.70	2	✓
Intel	Xeon E5-2680 v3	AVX2	256-bit	2	12	2.50	1	✗
Intel	Xeon Gold 6140	AVX-512BW	512-bit	2	18	2.30	2	✓
Intel	Xeon Gold 6142	AVX-512BW	512-bit	2	16	2.60	1	✗

https://doi.org/10.1016/j.softx.2019.100345

Multithreading : two main use cases

- Parallel processing
  - For computationally intensive tasks
  - That can be parallelized
  - Example: *n-body*, *video games*
- Request-Level Processing
  - To manage multiple heterogeneous tasks
  - Different tasks, of different lengths
  - Example: *IO-based tasks*, *web browser*, *OS*

Cache hierarchy



- Cache Hierarchy
- SRAM vs DRAM

Multicore : SMP vs DSM

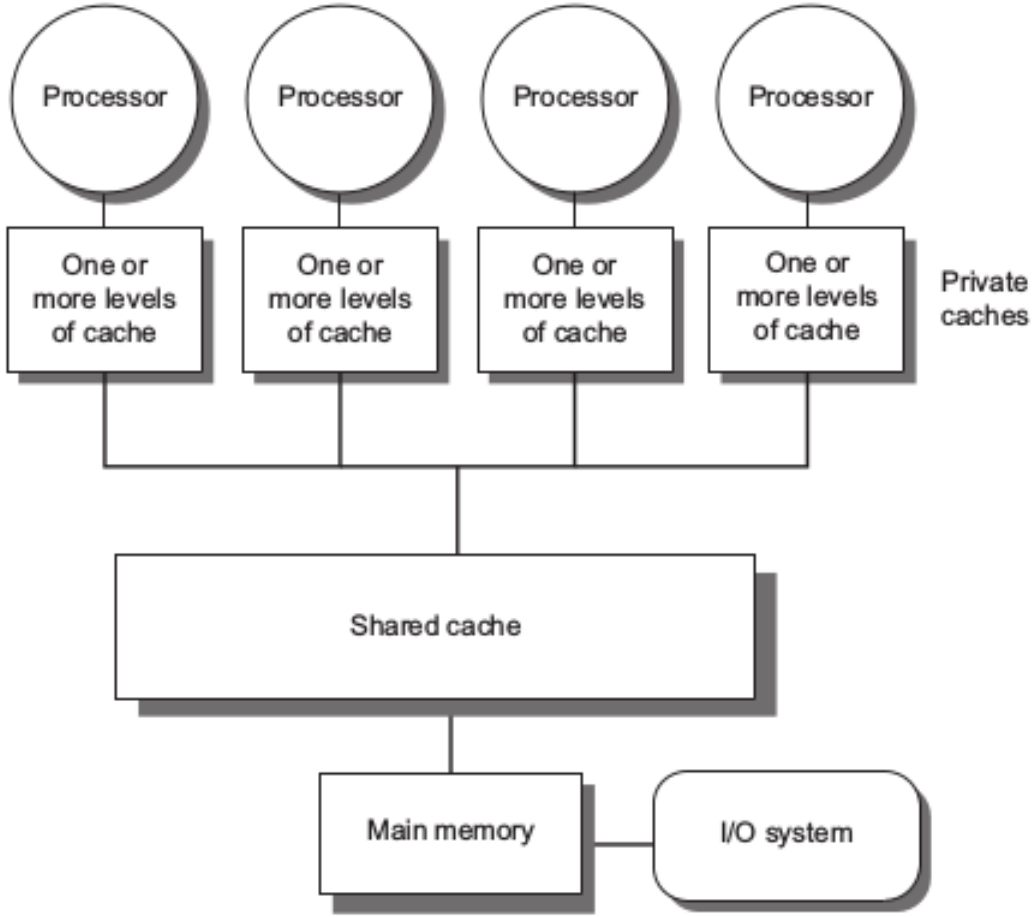


Figure: Symmetric MultiProcessor

Multicore : SMP vs DSM

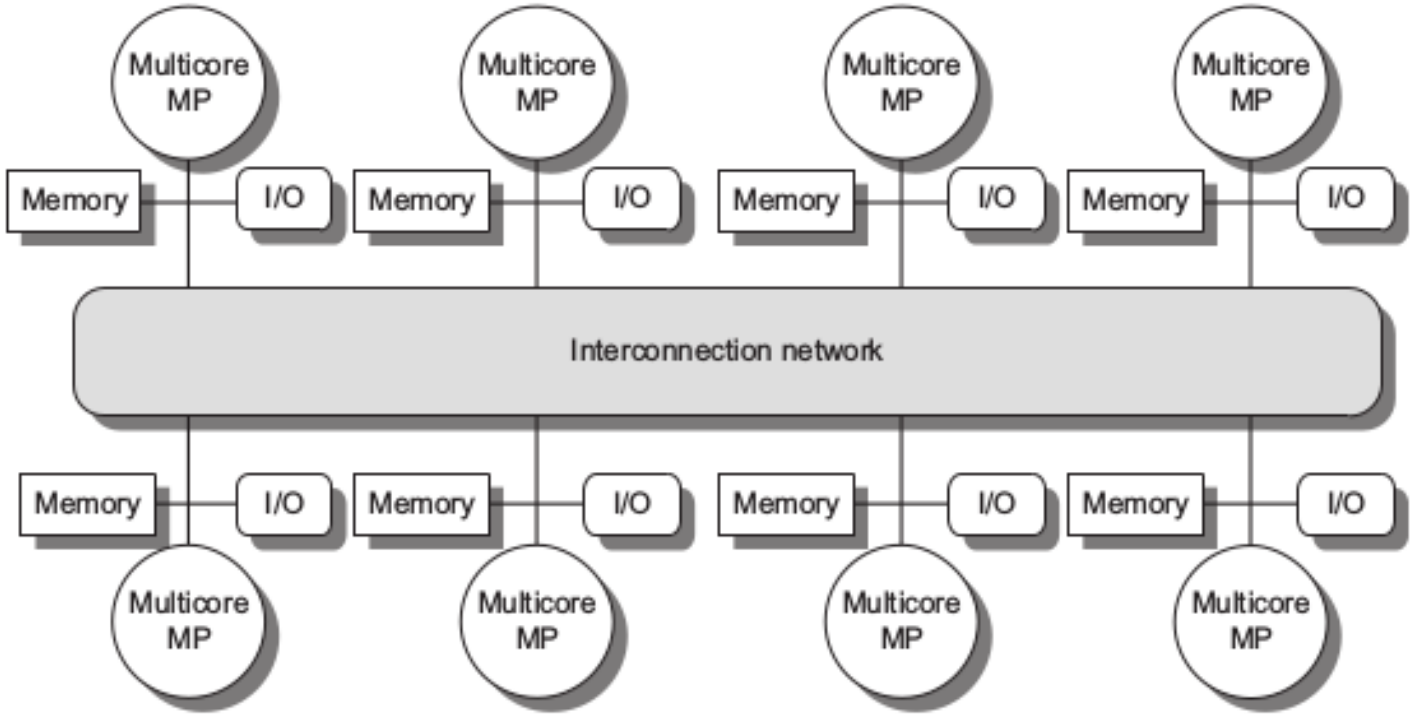
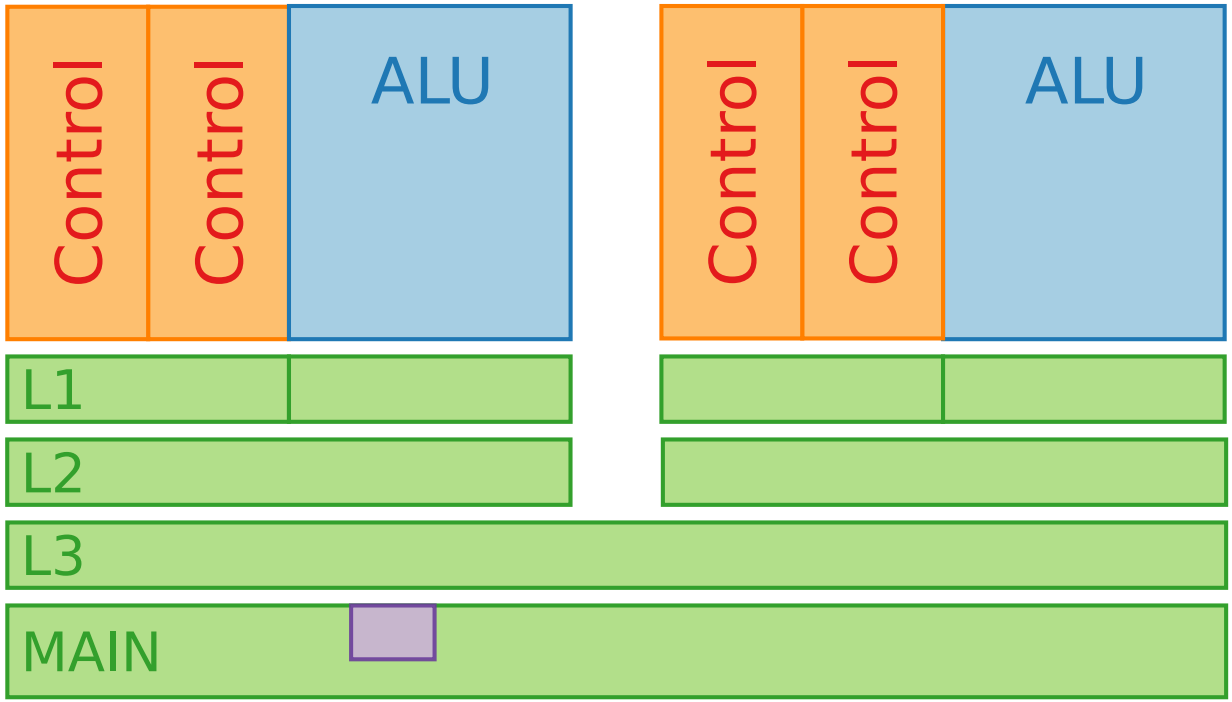
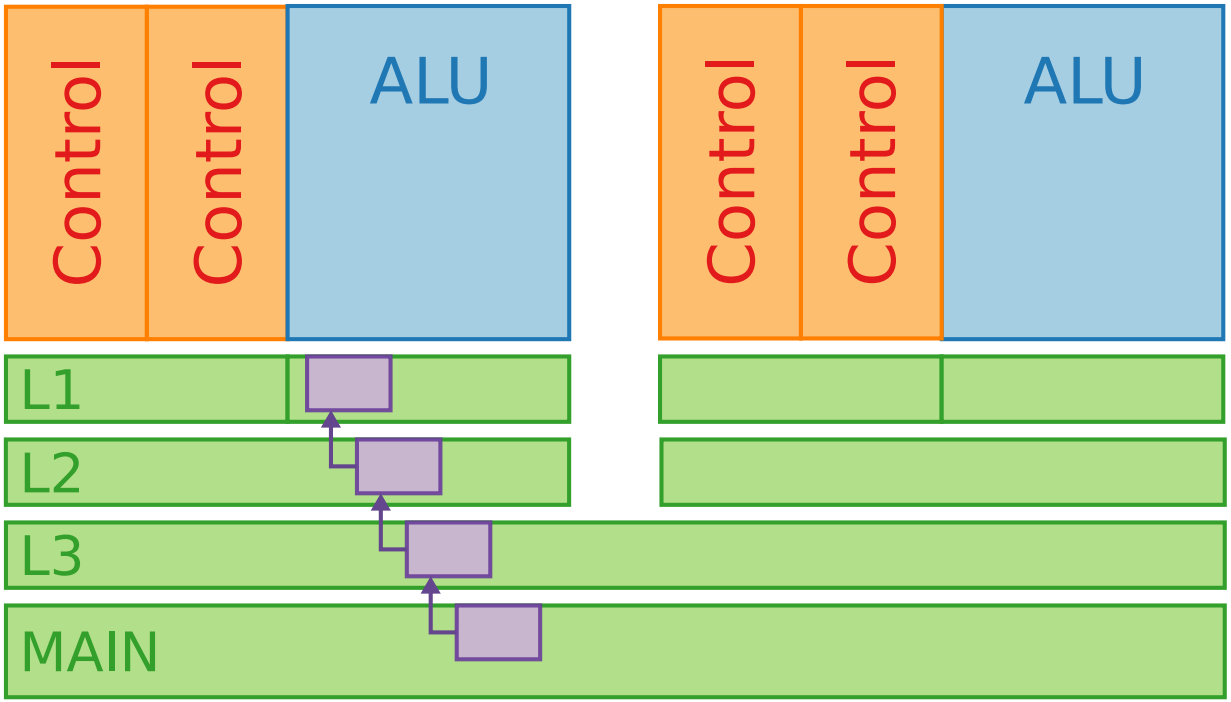


Figure: Distributed Shared Memory

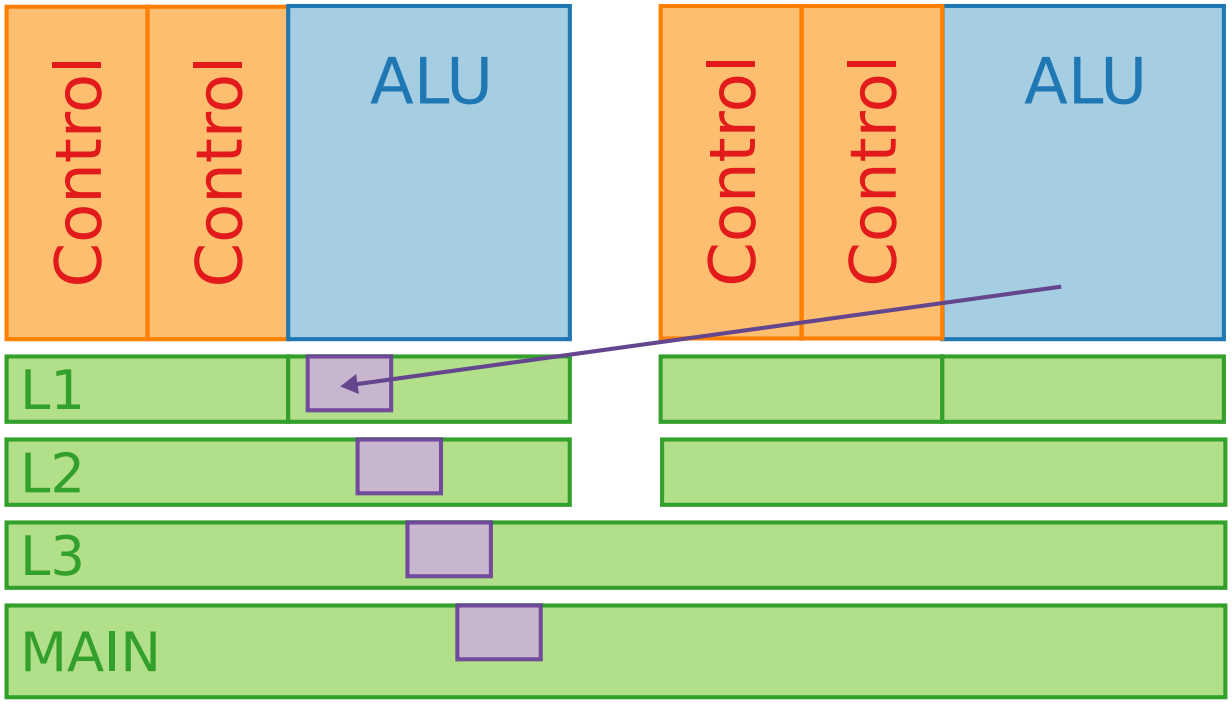
Multithreading : Cache coherence



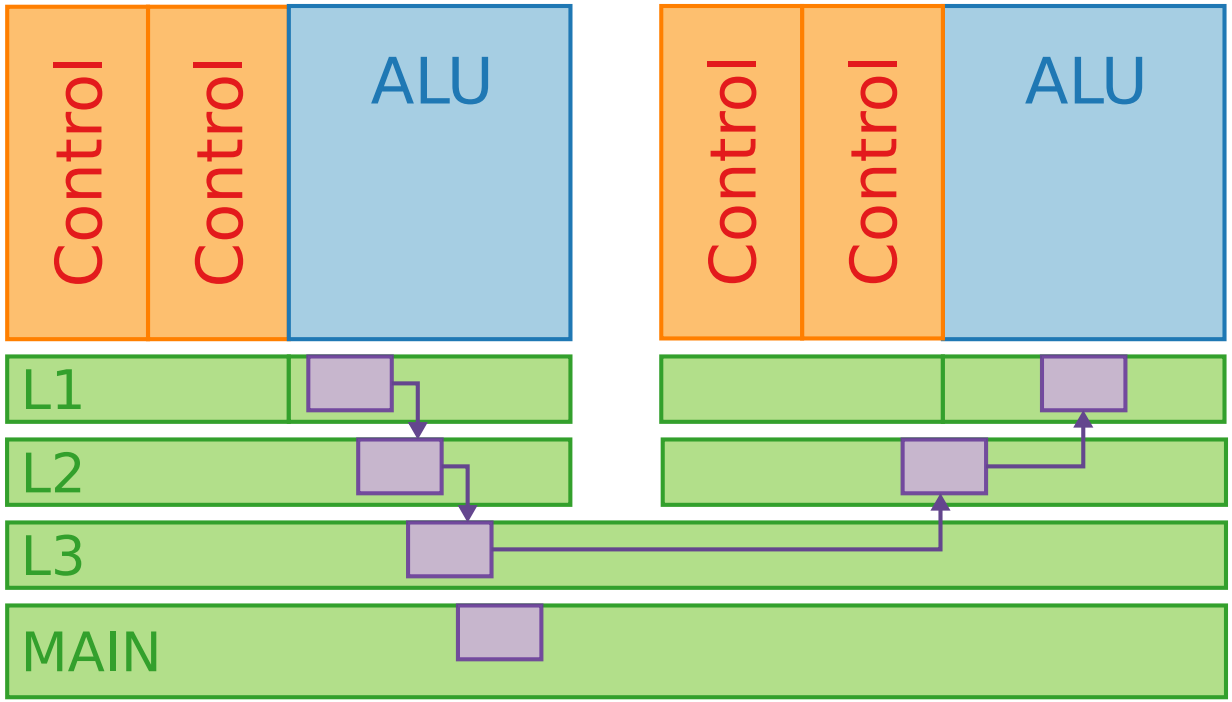
Multithreading : Cache coherence



Multithreading : Cache coherence



Multithreading : Cache coherence



Multithreading : Cache coherence

Ideally

- Any read must return most recent write
- Too strict to be implemented

Reality

- Any write must eventually be seen
- All writes are serialized (seen in the same order anywhere)

---

http://people.eecs.berkeley.edu/~pattrsn/252F96/Lecture18.pdf

Multithreading : Cache coherence

Ideally

- Any read must return most recent write
- Too strict to be implemented

Reality

- Any write must eventually be seen
- All writes are serialized (seen in the same order anywhere)

---

http://people.eecs.berkeley.edu/~pattrsn/252F96/Lecture18.pdf

Multithreading : Cache coherence

Ideally

- Any read must return most recent write
- Too strict to be implemented

Reality

- Any write must eventually be seen
- All writes are serialized (seen in the same order anywhere)

---

http://people.eecs.berkeley.edu/~pattrsn/252F96/Lecture18.pdf

Multithreading : Cache coherence

Ideally

- Any read must return most recent write
- Too strict to be implemented

Reality

- Any write must eventually be seen
- All writes are serialized (seen in the same order anywhere)

---

http://people.eecs.berkeley.edu/~pattrsn/252F96/Lecture18.pdf

Multithreading : Cache coherence

Two main approaches

- Snooping
  - Broadcast all requests to all processors
  - Each processor have a sharing status of each data
  - Typically used with bus (broadcast)
- Directory-based
  - Sharing status of data centralized
  - Point-to-point requests
  - Therefore, scales better

Multithreading : Cache coherence

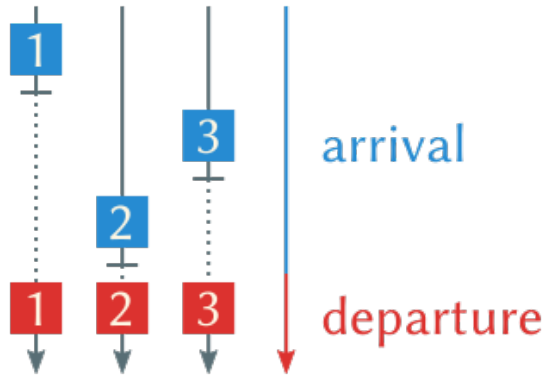
Two main approaches

- Snooping
  - Broadcast all requests to all processors
  - Each processor have a sharing status of each data
  - Typically used with bus (broadcast)
- Directory-based
  - Sharing status of data centralized
  - Point-to-point requests
  - Therefore, scales better

Need for Synchronization

- Cache coherence may suffice (*display*),
- Sometimes it does not, and we need to synchronize

- Mutex
  - Used to protect shared data from being simultaneously accessed by multiple threads
  - Owned by a single thread
- Semaphore
  - Acquired and released by any thread
- Barrier
  - All threads must reach the barrier before any can proceed
- Atomic operations
  - Read-modify-write operations



Tools

- C++
  - OpenMP
  - STL (std::threads)
- Other languages
  - Python: threading
  - Java: Runnable interface
  - ...